21

DEEP DIVE 21 · రెండు SYSTEMS, ఒక NETWORK

Design an ATM

> JPMorgan · Goldman · Visa · ప్రతి బ్యాంకు – పైకి ఇది Vending Machine లాగే కనిపిస్తుంది. **అది కాదు.**

మూడు విరుపులు. మొదటిది ₹3,000 **ఖాతా నుంచి తీసేసి నోట్లు ఇవ్వదు**. రెండోది ₹3,000 కోసం **₹9,000 తీసేసి** "విజయం" అంటుంది. మూడోది దాన్ని సరిచేయలేక – **ఆ విషయమే మర్చిపోతుంది.**

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి **ప్రతి output నిజంగా `node` lo run చేసినదే** – 2,000 ATMs, ₹1.96 కోట్ల అనుకరణతో సహా.

DEEP DIVE 20 (VENDING MACHINE) చదివారా? – ఇది దానికి కొనసాగింపు, పునరావృత్తం కాదు

Vending machine lo డబ్బు, నిల్వ, jam, rollback – అన్నీ చూశాం. మరియు అక్కడ rollback **పనిచేసింది**, ఎందుకంటే **అన్నీ ఒకే పెట్టెలో** ఉన్నాయి. `float`, `escrow`, `qty` – మూడింటినీ ఒక `clone()` తో వెనక్కి తిప్పగలం.

ATM lo **అది సాధ్యం కాదు**. ఖాతా నిల్వ ఒక **వేరే యంత్రంలో**, ఒక **network అవతల** ఉంది. ATM దాన్ని "వెనక్కి తిప్పలేదు" – అది కేవలం **అడగగలదు**, మరియు ఆ అభ్యర్థన కూడా విఫలం కావచ్చు.

ఈ ఒక్క తేడా మూడు కొత్త problems పుట్టిస్తుంది: idempotency, compensation, మరియు reconciliation. అవే ఈ doc.

(నోట్ల denominations లెక్క Deep Dive 20 §5 లోనిదే – దాన్ని ఇక్కడ మళ్ళీ రాబట్టను.)

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు network ఎందుకు అంతా మారుస్తుంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: "ఏమి జరిగిందో తెలియదు"

3. Step – Debit, ఆపై dispense
4. **మొదటి విరుపు** – మూడు వైఫల్యాలు, రెండు ఒకేలా కనిపిస్తాయి
5. Step – అనిశ్చితిని ఒక స్థితిగా గుర్తించడం

Part 3 – రెండో విరుపు: మళ్ళీ ప్రయత్నించడం

6. Step – "Timeout అంటే తాత్కాలికం, మళ్ళీ ప్రయత్నిద్దాం"
7. **రెండో విరుపు** – ₹3,000 కి ₹9,000 debit
8. Step – Idempotency key

Part 4 – మూడో విరుపు: సరిచేయలేకపోవడం

9. Step – నోట్లు ఇవ్వలేకపోతే వెనక్కి తిప్పడం
10. **మూడో విరుపు** – reversal కూడా విఫలమైంది, మరియు ఆ విషయం మాయమైంది
11. Step – Journal · మరియు తర్వాత సరిచేయడం

Part 5 – పూర్తి system

12. Step – Network ని తాకకముందే ఆపగలిగేవి
13. మొత్తం code · నడిపి చూద్దాం · **2,000 ATMs, ₹1.96 కోట్లు**

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటితో చెప్పాల్సిన English script
16. Follow-ups – రెండు ATMs, cash forecasting, EMV
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

---

1. అడిగింది ఏమిటి – మరియు network ఎందుకు అంతా మారుస్తుంది

> *"Design an ATM. A customer inserts a card, enters a PIN, chooses an amount, and the machine dispenses cash."*

ఒక్క పంక్తి తేడా · మొత్తం PROBLEM మారుతుంది

Vending machine vs ATM – పైకి ఒకటే, లోపల వేరే

Vending Machine (DD 20)	ATM
డబ్బు, నిల్వ, dispenser – అన్నీ **ఒకే పెట్టెలో**	నోట్లు ఇక్కడ · ఖాతా **అక్కడ** మధ్యలో ఒక network
jam అయితే: clone() → rollback ✓	jam అయితే: ఖాతాని **అడగాలి** ✗

మరియు "అడగడం" మూడు రకాలుగా ముగుస్తుంది

విజయం – జరిగింది, జవాబు వచ్చింది. · **Down** – చేరనేలేదు, ఏమీ జరగలేదు.

Timeout – చేరింది, జరిగింది, **కానీ జవాబు రాలేదు**.

ఆ మూడోది ఒక వైఫల్యం కాదు – అది ఒక **అనిశ్చితి**.

మరియు ఈ doc lo ఉన్న మూడు విరుపులూ ఆ ఒక్క పదం నుంచే పుడతాయి.

---

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఖాతా నిల్వ ATM lo ఉందా, bank server lo ఉందా?	**ఇదే మొత్తం problem** – server అంటే §4

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Network విఫలమైతే? Timeout అయితే?	§5 – మరియు ఈ రెండూ **వేర్వేరు** అని గుర్తించాలి
ఒక debit ని వెనక్కి తిప్పగలమా?	§9 – మరియు ఆ reversal కూడా విఫలం కావచ్చు (§10)
నోట్లు ఇవ్వలేకపోతే (jam)?	§9 – Deep Dive 20 §10 యొక్క అదే సమస్య, కానీ కష్టమైన సందర్భంలో
ఏ నోట్లు ఉన్నాయి? ₹1,350 ఇవ్వగలమా?	§12 – Deep Dive 20 §5 యొక్క DP
రోజువారీ పరిమితి? PIN ప్రయత్నాలు?	§12 – network ని తాకకముందే చేయగలిగేవి
ATM పునఃప్రారంభమైతే?	§11 – journal **తప్పనిసరి**

రెండో ప్రశ్న – మరియు ఇది INTERVIEW ని తెరుస్తుంది

"Bank కి పంపిన debit request timeout అయితే – ఆ debit జరిగిందా, జరగలేదా?"

సరైన జవాబు: **తెలియదు.** మరియు అదే మొత్తం ఇబ్బంది.

"Network down" అంటే request **చేరనేలేదు** – ఏమీ జరగలేదు, సురక్షితం.
"Timeout" అంటే request **చేరింది, server పని చేసింది, జవాబు తిరిగి రాలేదు** – లేదా అసలు చేరలేదు.
రెండూ ఒకేలా కనిపిస్తాయి.

ఈ ప్రశ్న అడిగితే, interviewer కి మీరు **distributed systems** ఆలోచిస్తున్నారని తెలుస్తుంది – ఒక్క machine ఆలోచన కాదని.

Part 2 — మొదటి విరుపు: "ఏమి జరిగిందో తెలియదు"

---

3. Step — Debit, ఆపై dispense

```
class ATM {
  constructor(bank, cash) { this.bank = bank; this.cash = cash; }
  withdraw(acct, amount, { dispenser = () => true } = {}) {
    const d = this.bank.debit(acct, amount);      // 1 · ఖాతా నుంచి తీసేయడం
    if (!d.ok) return d;
    this.cash -= amount;
    if (!dispenser(amount))
      return { ok: false, reason: 'DISPENSE_FAILED' };   // 2 · నోట్లు
    return { ok: true, dispensed: amount, balance: d.balance };
  }
}
```

క్రమం సరైనది — ముందు debit, ఆపై నోట్లు. (తిరగబడితే: నోట్లు ఇచ్చాక debit విఫలమైతే బ్యాంకు డబ్బు పోతుంది.)

```
ప్రారంభ నిల్వ  : ₹10000
₹2,000 తీస్తే  : { ok: true, dispensed: 2000, balance: 8000 }
₹50,000 తీస్తే : { ok: false, reason: 'INSUFFICIENT_FUNDS' }

ఇప్పుడు యంత్రం నోట్లు ఇవ్వలేకపోతే:
₹3,000 తీస్తే  : { ok: false, reason: 'DISPENSE_FAILED' }
ఖాతా నిల్వ     : ₹5000 ← డబ్బు పోయింది, నోట్లు రాలేదు
```

ఇది **Deep Dive 20 §10 యొక్క అదే సమస్య**, మరియు అక్కడి పరిష్కారం rollback. కానీ ఇక్కడ...

---

4. మొదటి విరుపు — మూడు వైఫల్యాలు, రెండు ఒకేలా కనిపిస్తాయి

Network ని నిజాయితీగా నమూనా చేద్దాం — **మూడు ముగింపులతో**:

```
class Network {
  constructor(script = []) { this.script = [...script]; }
  send(name, fn) {
    const mode = this.script.shift() ?? 'OK';
    if (mode === 'DOWN') return { ok: false, reason: 'NETWORK_DOWN' };  // చేరనేలేదు
    const result = fn();                                     // server చేసింది
    // చేరింది, జరిగింది — కానీ జవాబు రాలేదు
    if (mode === 'TIMEOUT') return { ok: false, reason: 'TIMEOUT' };
```

```
    return result;
  }
}
```

ఆ `fn()` ఎక్కడ ఉందో గమనించండి: `TIMEOUT` **lo అది ఇప్పటికే నడిచింది.** అదే నిజ ప్రపంచం.

```
అంతా బాగుంది
  ATM చూసినది : {"ok":true,"dispensed":3000,"balance":7000}
  ఖాతా నిల్వ   : ₹7000  ·  bank log: [debit A-1 ₹3000]

Network down (చేరలేదు)
  ATM చూసినది : {"ok":false,"reason":"NETWORK_DOWN"}
  ఖాతా నిల్వ   : ₹10000  ·  bank log: []

Timeout (చేరింది, జవాబు రాలేదు)
  ATM చూసినది : {"ok":false,"reason":"TIMEOUT"}
  ఖాతా నిల్వ   : ₹7000  ·  bank log: [debit A-1 ₹3000]
```

మొదటి విరుపు – ATM చూసే జవాబు నిజాన్ని దాచేస్తుంది

కింది రెండు వరుసలు చూడండి. **ATM కి రెండూ** `ok: false`.

కానీ ఖాతా నిల్వ ఒకదాంట్లో **₹10,000**, రెండోదాంట్లో **₹7,000**.

అంటే: **ATM కి తను చూస్తున్న వైఫల్యం ఏ రకమో తెలియదు.** ఏమీ జరగలేదా, లేక అంతా జరిగి జవాబు మాత్రం పోయిందా – ఆ తేడా ₹3,000.

మౌలిక తప్పు: `{ ok: false }` అనేది "జరగలేదు" అని అర్థం అనుకున్నాం. **Network అవతల అది "జరిగిందో లేదో తెలియదు" అని మాత్రమే అర్థం.** ఒక ఫలితం కాదు – ఒక **అజ్ఞానం**.

---

5. Step – అనిశ్చితిని ఒక స్థితిగా గుర్తించడం

ఈ సమస్యకి ఒక **తెలివైన పరిష్కారం లేదు**. ATM ఎప్పటికీ ఖచ్చితంగా తెలుసుకోలేదు.

చేయగలిగేది ఒక్కటే: **అనిశ్చితిని ఒక స్థితిగా అంగీకరించి, దాన్ని పరిష్కరించగలిగేలా చేయడం.** అది మూడు అవసరాలని పుట్టిస్తుంది –

అనిశ్చితి నుంచి పుట్టే మూడు అవసరాలు

"జరిగిందో లేదో తెలియదు" – దీన్ని సురక్షితంగా నిర్వహించడానికి

1 · Idempotency	2 · Compensation	3 · Reconciliation
మళ్ళీ అడిగితే రెండోసారి జరగకూడదు	వెనక్కి తిప్పలేను, కానీ **ఎదురు పని** చేయగలను	ఇప్పుడు కుదరకపోతే **తర్వాత** కుదరాలి
→ §8	→ §9	→ §11

ఈ మూడూ కలిపి ఒక పేరు ఉంది: **saga** – పలు systems మీద విస్తరించిన లావాదేవీ.

ఒక database transaction కాదు; ఒక **దశల వరుస**, ప్రతి దశకీ ఒక ఎదురు దశ.

ATM ఆ నమూనా యొక్క **అతి స్పష్టమైన ఉదాహరణ** – అందుకే ఇది ఒక మంచి interview problem.

Part 3 – రెండో విరుపు: మళ్ళీ ప్రయత్నించడం

6. Step – "Timeout అంటే తాత్కాలికం, మళ్ళీ ప్రయత్నిద్దాం"

Timeout ఒక తాత్కాలిక సమస్యలా అనిపిస్తుంది. Network కొంచెం నెమ్మదైంది, అంతే. **మళ్ళీ ప్రయత్నిద్దాం** – ఇది ప్రతి HTTP client lo ఉండే ప్రవర్తన:

```
withdraw(acct, amount) {
  for (let attempt = 0; attempt <= this.retries; attempt++) {
    const r = this.net.send(() => this.bank.debit(acct, amount));
    if (r.ok) return { ok: true, dispensed: amount,
                       balance: r.balance, attempts: attempt+1 };
    // ఇది మళ్ళీ ప్రయత్నించవద్దు
    if (r.reason === 'INSUFFICIENT_FUNDS') return r;
  }
  return { ok: false, reason: 'GAVE_UP', attempts: this.retries + 1 };
}
```

ఇది **జాగ్రత్తగా రాసిన code** – `INSUFFICIENT_FUNDS` ని మళ్ళీ ప్రయత్నించట్లేదు, ప్రయత్నాల సంఖ్యని తిరిగి ఇస్తోంది.

7. రెండో విరుపు – ₹3,000 కి ₹9,000 debit

```
Customer ₹3,000 అడిగాడు. మొదటి రెండు ప్రయత్నాలు timeout అయ్యాయి.

  ATM చూసినది   : { ok: true, dispensed: 3000, balance: 1000, attempts: 3 }
  నోట్లు ఇచ్చినవి : ₹3,000  (ఒక్కసారే)
  bank debits    : ₹3000 + ₹3000 + ₹3000 =  ₹9000
  ఖాతా నిల్వ     : ₹1000 ← ₹9,000 పోయింది, ₹3,000 వచ్చింది
```

రెండో విరుపు – ATM "విజయం" అని చెప్పింది, మరియు అది నిజమే

Customer ₹3,000 అడిగాడు. అతనికి ₹3,000 వచ్చాయి. ATM screen "Transaction successful" చూపించింది. **అంతా సరిగ్గా జరిగినట్టే కనిపిస్తోంది.**

కానీ అతని ఖాతా నుంచి **₹9,000** పోయాయి – ఎందుకంటే మొదటి రెండు "విఫలమైన" ప్రయత్నాలూ **నిజంగా విజయవంతమయ్యాయి**, జవాబు మాత్రమే పోయింది.

మరియు ఎవరికీ తెలియదు. ATM కి తెలియదు, customer కి వెంటనే తెలియదు, bank కి మూడు చెల్లుబాటయ్యే debits కనిపిస్తాయి.

మౌలిక తప్పు: "మళ్ళీ ప్రయత్నించడం" అనేది **ఒక operation మళ్ళీ చేస్తే అదే ఫలితం వస్తుంది** అని ఊహిస్తుంది. చదవడానికి అది నిజం. **డబ్బు తీసేయడానికి అది పూర్తిగా అబద్ధం.**

---

8. Step – Idempotency key

పరిష్కారం: **ప్రతి కోరికకీ ఒక ప్రత్యేక గుర్తింపు**, మరియు bank ఆ గుర్తింపుతో ఫలితాన్ని గుర్తుపెట్టుకుంటుంది.

```
debit(key, acct, amount) {
  if (this.seen.has(key))
    return { ...this.seen.get(key), replayed: true };   // ఇప్పటికే చేసినది
  ...
  this.seen.set(key, res);                             // ఫలితాన్ని key తో దాచడం
  return res;
}
```

మరియు ATM వైపు – **ఒక్కో కోరికకి ఒక్కటే key, ప్రయత్నాలన్నిటికీ అదే**:

```
withdraw(acct, amount) {
  const key = `W-${++this.#seq}`;         // ఒక్కో కోరికకి ఒక్కటే key
  for (let attempt = 0; attempt <= this.retries; attempt++) {
    const r = this.net.send(() => this.bank.debit(key, acct, amount));
    ...
  }
}
```

ఆ `KEY` ఎక్కడ పుడుతుందో అదే కీలకం

ఇది **ATM lo** పుడుతుంది, loop కి **బయట**. మూడు ప్రయత్నాలూ **అదే key** పంపుతాయి.

Bank కి "ఇది మూడో ప్రయత్నమా" అని తెలియనవసరం లేదు – అది **key ని చూసి** "ఇది ఇప్పటికే చేశాను" అని గుర్తిస్తుంది, మరియు **అదే ఫలితాన్ని** తిరిగి ఇస్తుంది.

నియమం: *Idempotency అనేది ఒక operation యొక్క లక్షణం కాదు – అది ఒక **ఒప్పందం**.* పంపేవాడు ఒక స్థిరమైన key ఇవ్వాలి; అందుకునేవాడు దాన్ని గుర్తుపెట్టుకోవాలి. ఒక్కరు తప్పినా అది విరిగిపోతుంది.

```
అదే పరిస్థితి — రెండు timeouts, ఆపై విజయం

  ATM చూసినది   : { ok: true, dispensed: 3000, balance: 7000,
                    attempts: 3, replayed: true }
  bank debits    : ₹3000
  ఖాతా నిల్వ     : ₹7000 ← సరిగ్గా ₹3,000 మాత్రమే ✓
```

మూడు ప్రయత్నాలు, ఒక్క debit. మరియు `replayed: true` – ATM కి "ఇది ఒక పునరావృత్తి" అని తెలుసు.

Part 4 – మూడో విరుపు: సరిచేయలేకపోవడం

9. Step – నోట్లు ఇవ్వలేకపోతే వెనక్కి తిప్పడం

Debit విజయవంతమైంది. ఇప్పుడు నోట్లు ఇరుక్కున్నాయి. **ఎదురు పని చేయాలి** – ఒక `reverse`:

```
reverse(key, acct, amount) {
  const rk = 'R:' + key;
  if (this.seen.has(rk)) return { ...this.seen.get(rk), replayed: true };
  const b = this.accounts.get(acct);
  this.accounts.set(acct, b + amount);
  ...
}
```

Reversal కూడా idempotent కావాలి – అదే key తో, ఒక `R:` ఉపసర్గతో. లేకపోతే §7 యొక్క సమస్య **అద్దంలో** తిరిగి వస్తుంది: reversal ని మూడుసార్లు retry చేస్తే customer కి ₹6,000 అదనంగా వస్తాయి.

మరియు గమనించండి – ఇది **rollback కాదు**. Rollback అంటే "జరగనట్టు చేయడం"; ledger lo మొదటి debit **ఇప్పటికీ ఉంటుంది**, దాని పక్కన ఒక ఎదురు entry చేరుతుంది. **అదే compensation.**

10. మూడో విరుపు – reversal కూడా విఫలమైంది, మరియు ఆ విషయం మాయమైంది

```
నోట్లు ఇవ్వలేకపోయాం — reversal ప్రయత్నిస్తాం

  reversal విజయవంతం
    ATM      : {"ok":false,"reason":"DISPENSE_FAILED","reversed":true}
    bank log : −₹3000 +₹3000  ·  నిల్వ ₹10000

  reversal కూడా విఫలం
    ATM      : {"ok":false,"reason":"DISPENSE_FAILED","reversed":false,
               "reversalError":"NETWORK_DOWN"}
    bank log : −₹3000  ·  నిల్వ ₹7000
```

మూడో విరుపు – ATM కి తెలుసు, కానీ ఆ జ్ఞానం ఎక్కడికీ వెళ్ళదు

రెండో సందర్భంలో customer **₹3,000 పోగొట్టుకున్నాడు**. నోట్లు రాలేదు. ATM కి ఇది **తెలుసు** – `reversed: false` అని తిరిగి ఇస్తోంది.

కానీ ఆ function return అయిన క్షణం, ఆ జ్ఞానం మాయమవుతుంది.

ఎవరూ దాన్ని దాచలేదు. ఎవరూ మళ్ళీ ప్రయత్నించరు. ATM పునఃప్రారంభమైతే ఆ విషయం ఉనికిలోనే ఉండదు. Customer బ్యాంకుకి ఫోన్ చేసి, నిరూపించుకుని, వారాల తర్వాత తన డబ్బు తిరిగి పొందాలి.

మౌలిక తప్పు: మనం "ఇప్పుడే సరిచేయాలి" అని ప్రయత్నించాం. కానీ network విఫలమైనప్పుడు **ఇప్పుడు అనేది ఉండదు**. కావాల్సినది: **సరిచేయాల్సిన పని ఒక శాశ్వత రికార్డుగా మిగలాలి.**

---

11. Step – Journal · మరియు తర్వాత సరిచేయడం

ATM ప్రతి లావాదేవీని **ముందు రాస్తుంది, ఆపై చేస్తుంది** – మరియు ప్రతి దశనీ గుర్తిస్తుంది:

```
const PHASE = {
  STARTED: 'STARTED', DEBITED: 'DEBITED', DISPENSED: 'DISPENSED',
  COMPLETE: 'COMPLETE', REVERSAL_PENDING: 'REVERSAL_PENDING',
  REVERSED: 'REVERSED', DECLINED: 'DECLINED',
};
class Journal {
  entries = new Map();
  open(key, acct, amount) {
    const e = { key, acct, amount, phase: PHASE.STARTED, at: this.entries.size };
    this.entries.set(key, e);
    return e;
  }
  mark(key, phase, extra = {}) {
    const e = this.entries.get(key);
    Object.assign(e, { phase }, extra);
    return e;
  }
  get unresolved() {
    return [...this.entries.values()]
             .filter(e => e.phase === PHASE.REVERSAL_PENDING);
  }
}
```

మరియు **తర్వాత సరిచేయడం**:

```
// ---- మిగిలిపోయినవాటిని మళ్ళీ ప్రయత్నించడం ----
reconcile() {
  const pending = this.journal.unresolved;
  let fixed = 0;
  for (const e of pending) if (this.#tryReverse(e.key)) fixed++;
  return { checked: pending.length, resolved: fixed,
```

```
            stillPending: this.journal.unresolved.length };
}
```

TIMEOUT కూడా `REVERSAL_PENDING` కి వెళుతుంది – మరియు అది ఉద్దేశపూర్వకం

Debit timeout అయితే, debit జరిగిందో లేదో తెలియదు. ATM ఏమి చేయాలి?

జవాబు: reversal పంపాలి. Debit జరిగి ఉంటే అది సరిచేస్తుంది; జరగకపోతే bank `NOTHING_TO_REVERSE` అంటుంది, అదీ పర్వాలేదు.

`if (d.reason === 'TIMEOUT') { mark(REVERSAL_PENDING); tryReverse(key); }`

ఇది ఒక ముఖ్యమైన వైఖరి: అనిశ్చితిని ఎదుర్కొన్నప్పుడు, **సురక్షితమైన వైపు తప్పు చేయడం**. Customer కి నోట్లు రాలేదు కాబట్టి, debit ఉండకూడదు. Reversal అనవసరమైనా హాని లేదు (idempotent); reversal చేయకపోతే customer డబ్బు పోతుంది.

```
--- §10: reversal కూడా విఫలం → journal lo మిగిలింది ---
  ₹3,000 → { ok: false, reason: 'DISPENSE_FAILED', key: 'W-0001',
             reversed: false, willReconcile: true }
  నిల్వ         : ₹17000 ← ఇంకా తక్కువే
  journal       : W-0001=REVERSAL_PENDING

  తర్వాత network తిరిగి వచ్చింది. reconcile():
    { checked: 1, resolved: 1, stillPending: 0 }
    నిల్వ ఇప్పుడు : ₹20000 ← సరిచేయబడింది ✓
    journal       : W-0001=REVERSED
```

Part 5 – పూర్తి system

12. Step – Network ని తాకకముందే ఆపగలిగేవి

ఒక చివరి ఆలోచన, మరియు ఇది సులభమైనది కానీ ముఖ్యమైనది: **network ని అడగకుండానే తెలిసిపోయేవి ముందే తనిఖీ చేయాలి.**

```
// ---- ముందు, network ని తాకకుండా చేయగలిగే తనిఖీలు ----
if (amount <= 0) return { ok: false, reason: `BAD_AMOUNT: ${amount}` };
const today = this.withdrawnToday.get(acct) ?? 0;
if (today + amount > this.dailyLimit)
  return { ok: false,
           reason: `DAILY_LIMIT: ₹${this.dailyLimit - today} మిగిలింది` };
const plan = this.cash.plan(amount);
if (!plan) return { ok: false, reason: `CANNOT_DISPENSE: ₹${amount}` };

// ---- ఇక్కడి నుంచి journal lo రాస్తున్నాం ----
const key = `W-${String(++this.#seq).padStart(4, '0')}`;
this.journal.open(key, acct, amount);
```

```
--- తిరస్కరణలు (network తాకకుండా) ---
  నోట్లతో ఇవ్వలేనిది : { ok: false, reason: 'CANNOT_DISPENSE: ₹1350' }
  రోజువారీ పరిమితి   : { ok: false, reason: 'DAILY_LIMIT: ₹25000 మిగిలింది' }
  ఋణాత్మకం          : { ok: false, reason: 'BAD_AMOUNT: -100' }
  network calls      : 0 ← సున్నా, అన్నీ ముందే ఆగిపోయాయి
```

₹1,350 ని ఎందుకు ముందే ఆపాలి

ATM lo ₹100, ₹200, ₹500, ₹2000 నోట్లు ఉన్నాయి. **₹1,350 ఇవ్వడం అసాధ్యం** – ₹50 నోట్లు లేవు.

దీన్ని **debit తర్వాత** కనుక్కుంటే, ఒక అనవసరమైన debit + reversal జంట ledger lo మిగులుతుంది, ఒక network round-trip వృథా, మరియు ఒక reversal విఫలమయ్యే అవకాశం పుడుతుంది.

నియమం: ప్రమాదకరమైన దశకి ముందు, ఖచ్చితంగా చెప్పగలిగేవన్నీ చెప్పేయాలి. ఇది Deep Dive 20 §12 (`canBuy`) యొక్క అదే ఆలోచన – కానీ ఇక్కడ దాని విలువ చాలా ఎక్కువ, ఎందుకంటే తర్వాతి దశ **వెనక్కి తీసుకోలేనిది**.

13. మొత్తం code · నడిపి చూద్దాం · 2,000 ATMs, ₹1.96 కోట్లు

నిర్మాణం · నాలుగు ముక్కలు, ఒక NETWORK

ATM కి ఖాతా కనిపించదు – అది ఒక network అవతల

అచలం: **ఖాతా నుంచి పోయినది = ఇచ్చిన నగదు + పరిష్కారం కాని బాకీలు**

అది ఎప్పుడూ నిజం – timeouts lo నూ, jams lo నూ, reconcile ముందూ తర్వాతా.

```
'use strict';
class Bank {
  #accounts = new Map();
  #seen = new Map();                              // idempotency key → ఫలితం
  ledger = [];
  constructor(accounts) {
    for (const [a, b] of Object.entries(accounts)) this.#accounts.set(a, b);
  }
  balance(acct) { return this.#accounts.get(acct); }

  debit(key, acct, amount) {
    if (this.#seen.has(key)) return { ...this.#seen.get(key), replayed: true };
    let res;
    const bal = this.#accounts.get(acct);
    if (bal === undefined) res = { ok: false, reason: `NO_ACCOUNT: ${acct}` };
    else if (amount <= 0) res = { ok: false, reason: `BAD_AMOUNT: ${amount}` };
    else if (bal < amount)
      res = { ok: false, reason: `INSUFFICIENT_FUNDS: ₹${bal}` };
    else {
      this.#accounts.set(acct, bal - amount);
      this.ledger.push({ key, acct, delta: -amount });
      res = { ok: true, balance: bal - amount };
    }
    this.#seen.set(key, res);
    return res;
  }
  // ఒక debit ని వెనక్కి తిప్పడం – అదే key తో, ఒక్కసారే
  reverse(key, acct, amount) {
    const rk = `R:${key}`;
    if (this.#seen.has(rk)) return { ...this.#seen.get(rk), replayed: true };
    if (!this.#seen.has(key) || !this.#seen.get(key).ok)
      return { ok: false, reason: `NOTHING_TO_REVERSE: ${key}` };
    const bal = this.#accounts.get(acct);
    this.#accounts.set(acct, bal + amount);
    this.ledger.push({ key: rk, acct, delta: +amount });
    const res = { ok: true, balance: bal + amount };
    this.#seen.set(rk, res);
    return res;
  }
}
```

```
// చిల్లర లెక్క Deep Dive 20 §5 లోనిదే — అదే bounded coin change DP
class CashBox {
  #n = new Map();
  constructor(init = {}) {
    for (const [k,v] of Object.entries(init)) this.#n.set(+k, v);
  }
  get denoms() { return [...this.#n.keys()].sort((a,b) => b-a); }
  count(d) { return this.#n.get(d) ?? 0; }
  add(d, n = 1) { this.#n.set(d, this.count(d) + n); return this; }
  get total() { return this.denoms.reduce((s,d) => s + d*this.count(d), 0); }
  clone() {
    const c = new CashBox();
    for (const d of this.denoms) c.add(d, this.count(d));
    return c;
  }
  toString() {
    const p = this.denoms.filter(d => this.count(d))
                  .map(d => `${this.count(d)}×₹${d}`);
    return p.length ? p.join(' ') : 'ఖాళీ';
  }
  plan(amount) {                                // ఈ మొత్తాన్ని ఇవ్వగలమా
    if (amount === 0) return new CashBox();
    const best = new Array(amount + 1).fill(null);
    best[0] = new Map();
    for (const d of this.denoms) {
      const avail = this.count(d);
      for (let take = 1; take <= avail; take++)
        for (let v = amount; v >= d; v--)
          if (best[v] === null && best[v-d] !== null &&
              (best[v-d].get(d) ?? 0) < avail) {
            const m = new Map(best[v-d]);
            m.set(d, (m.get(d) ?? 0) + 1); best[v] = m;
          }
    }
    if (best[amount] === null) return null;
    const out = new CashBox();
    for (const [d, n] of best[amount]) out.add(d, n);
    return out;
  }
  take(plan) {
    for (const d of plan.denoms) {
      if (this.count(d) < plan.count(d)) throw new Error(`NOT_ENOUGH_NOTES: ₹${d}`);
      this.#n.set(d, this.count(d) - plan.count(d));
    }
    return this;
  }
}

const PHASE = {
  STARTED: 'STARTED', DEBITED: 'DEBITED', DISPENSED: 'DISPENSED',
  COMPLETE: 'COMPLETE', REVERSAL_PENDING: 'REVERSAL_PENDING',
  REVERSED: 'REVERSED', DECLINED: 'DECLINED',
};
class Journal {
  entries = new Map();
  open(key, acct, amount) {
    const e = { key, acct, amount, phase: PHASE.STARTED, at: this.entries.size };
    this.entries.set(key, e);
```

```
    return e;
  }
  mark(key, phase, extra = {}) {
    const e = this.entries.get(key);
    Object.assign(e, { phase }, extra);
    return e;
  }
  get unresolved() {
    return [...this.entries.values()]
             .filter(e => e.phase === PHASE.REVERSAL_PENDING);
  }
}

class ATM {
  #seq = 0;
  constructor(bank, net, cash, { dailyLimit = 25_000, maxPin = 3 } = {}) {
    Object.assign(this, { bank, net, cash: new CashBox(cash), dailyLimit, maxPin });
    this.journal = new Journal();
    this.pinFails = new Map();
    this.withdrawnToday = new Map();
  }
  #call(fn) { return this.net.send(fn); }

  authenticate(card, pin, correctPin) {
    const fails = this.pinFails.get(card) ?? 0;
    if (fails >= this.maxPin) return { ok: false, reason: 'CARD_BLOCKED' };
    if (pin !== correctPin) {
      this.pinFails.set(card, fails + 1);
      return { ok: false, reason: `WRONG_PIN`, left: this.maxPin - fails - 1 };
    }
    this.pinFails.delete(card);
    return { ok: true };
  }

  withdraw(acct, amount, { dispenser = () => true } = {}) {
    // ---- ముందు, network ని తాకకుండా చేయగలిగే తనిఖీలు ----
    if (amount <= 0) return { ok: false, reason: `BAD_AMOUNT: ${amount}` };
    const today = this.withdrawnToday.get(acct) ?? 0;
    if (today + amount > this.dailyLimit)
      return { ok: false,
           reason: `DAILY_LIMIT: ₹${this.dailyLimit - today} మిగిలింది` };
    const plan = this.cash.plan(amount);
    if (!plan) return { ok: false, reason: `CANNOT_DISPENSE: ₹${amount}` };

    const key = `W-${String(++this.#seq).padStart(4, '0')}`;
    this.journal.open(key, acct, amount);

    const d = this.#call(() => this.bank.debit(key, acct, amount));
    if (!d.ok) {
      // TIMEOUT అయితే debit జరిగిందో లేదో తెలియదు — reversal కి పంపడం సురక్షితం
      if (d.reason === 'TIMEOUT') {
        this.journal.mark(key, PHASE.REVERSAL_PENDING, { why: 'DEBIT_UNCERTAIN' });
        this.#tryReverse(key);
        return { ok: false, reason: 'TIMEOUT', key, willReconcile: true };
      }
      this.journal.mark(key, PHASE.DECLINED, { why: d.reason });
      return { ok: false, reason: d.reason, key };
    }
    this.journal.mark(key, PHASE.DEBITED);
```

```
    this.cash.take(plan);
    if (!dispenser(plan)) {
      // నోట్లు వెనక్కి పెట్టెలోకి
      for (const dd of plan.denoms) this.cash.add(dd, plan.count(dd));
      this.journal.mark(key, PHASE.REVERSAL_PENDING, { why: 'DISPENSE_FAILED' });
      const rev = this.#tryReverse(key);
      return { ok: false, reason: 'DISPENSE_FAILED', key,
               reversed: rev, willReconcile: !rev };
    }
    this.journal.mark(key, PHASE.COMPLETE);
    this.withdrawnToday.set(acct, today + amount);
    return { ok: true, key, dispensed: plan.toString(),
             amount, balance: d.balance };
  }

  #tryReverse(key) {
    const e = this.journal.entries.get(key);
    const r = this.#call(() => this.bank.reverse(key, e.acct, e.amount));
    if (r.ok || r.reason?.startsWith('NOTHING_TO_REVERSE')) {
      this.journal.mark(key, PHASE.REVERSED);
      return true;
    }
    return false;                                // REVERSAL_PENDING గానే ఉంటుంది
  }

  // ---- మిగిలిపోయినవాటిని మళ్ళీ ప్రయత్నించడం ----
  reconcile() {
    const pending = this.journal.unresolved;
    let fixed = 0;
    for (const e of pending) if (this.#tryReverse(e.key)) fixed++;
    return { checked: pending.length, resolved: fixed,
             stillPending: this.journal.unresolved.length };
  }
}
```

```
--- సాధారణ withdrawal ---
  నోట్ల పెట్టె : 5×₹2000 10×₹500 10×₹200 20×₹100
  ₹3,700 → { ok: true, key: 'W-0001', dispensed: '1×₹2000 3×₹500 1×₹200', amount: 3700, balance: 16300 }
  నోట్ల పెట్టె : 4×₹2000 7×₹500 9×₹200 20×₹100
  ఖాతా నిల్వ   : ₹16300

--- §7: retry double-debit ఇక లేదు ---
  ₹3,000 → { ok: false, reason: 'TIMEOUT', key: 'W-0001', willReconcile: true }
  ledger  : W-0001 -3000 · R:W-0001 +3000
  నిల్వ    : ₹20000 ← debit జరిగింది, వెంటనే వెనక్కి ✓

--- §10: dispense విఫలం, reversal విజయం ---
  ₹3,000 → { ok: false, reason: 'DISPENSE_FAILED', key: 'W-0001', reversed: true, willReconcile: false }
  నిల్వ    : ₹20000
  నోట్లు   : 5×₹2000 10×₹500 10×₹200 20×₹100

--- §10: reversal కూడా విఫలం → journal lo మిగిలింది ---
  ₹3,000 → { ok: false, reason: 'DISPENSE_FAILED', key: 'W-0001', reversed: false, willReconcile: true }
```

```
  నిల్వ          : ₹17000 ← ఇంకా తక్కువే
  journal       : W-0001=REVERSAL_PENDING

  తర్వాత network తిరిగి వచ్చింది. reconcile():
    { checked: 1, resolved: 1, stillPending: 0 }
    నిల్వ ఇప్పుడు : ₹20000 ← సరిచేయబడింది ✓
    journal       : W-0001=REVERSED

--- తిరస్కరణలు (network తాకకుండా) ---
  నోట్లతో ఇవ్వలేనిది : { ok: false, reason: 'CANNOT_DISPENSE: ₹1350' }
  రోజువారీ పరిమితి   : { ok: false, reason: 'DAILY_LIMIT: ₹25000 మిగిలింది' }
  ఋణాత్మకం          : { ok: false, reason: 'BAD_AMOUNT: -100' }
  network calls      : 0 ← సున్నా, అన్నీ ముందే ఆగిపోయాయి

--- ఖాతా / PIN ---
  సరిపోని నిల్వ : { ok: false, reason: 'INSUFFICIENT_FUNDS: ₹1000', key: 'W-0001' }
  PIN 1111       : { ok: false, reason: 'WRONG_PIN', left: 2 }
  PIN 2222       : { ok: false, reason: 'WRONG_PIN', left: 1 }
  PIN 3333       : { ok: false, reason: 'WRONG_PIN', left: 0 }
  PIN 1234       : { ok: false, reason: 'CARD_BLOCKED' }
```

అసలు రుజువు – రెండు systems మీద డబ్బు సంరక్షణ

Vending machine lo ఒక్క సమీకరణం సరిపోయింది (DD 20 §13). ఇక్కడ **రెండు systems** ఉన్నాయి, కాబట్టి సమీకరణం పెద్దది:

> **ఖాతా నుంచి పోయినది = ఇచ్చిన నగదు + పరిష్కారం కాని బాకీలు**

2,000 ATMs, ఒక్కొక్కటీ 8 యాదృచ్ఛిక withdrawals, **network lo ప్రతి call కి 25% వైఫల్య అవకాశం** (timeouts మరియు outages కలిపి), 20% jams:

```
2,000 ATMs · 9555 పూర్తి · 4516 వెనక్కి · 0 పరిష్కారం కానివి
మొత్తం ఇచ్చిన నగదు: ₹1,96,60,100
ledger ↔ నిల్వ ↔ నోట్లు ↔ journal — అన్నీ ఏకీభవిస్తున్నాయా? అవును ✓
```

నాలుగు అచలాలు: **ledger మొత్తం = నిల్వ మార్పు**, **నోట్ల పెట్టె తగ్గుదల = ఇచ్చిన నగదు**, **ఖాతా తగ్గుదల = నగదు + బాకీలు**, మరియు **ప్రతి journal entry ఒక చెల్లుబాటయ్యే phase lo**.

మరియు network తిరిగి వచ్చాక `reconcile()` – 0 పరిష్కారం కానివి. ₹1.96 కోట్లు, 4,516 reversals, ఒక్క రూపాయి కూడా ఇరుక్కుపోలేదు.

ఒక పద్ధతి పొరపాటు, నేను చేసినది: ఈ పరీక్ష మొదట **206 వైఫల్యాలు** చూపించింది. నేను వెంటనే `#tryReverse` ని అనుమానించాను.

నిజానికి ledger పూర్తిగా సరైనది – నా **పరీక్ష** తప్పు. నేను "network తిరిగి వచ్చింది" అని చెప్పడానికి `script.push('OK')` చేశాను, కానీ `send()` script ని **ముందు నుంచి** తీసుకుంటుంది. ఆ 'OK' లు జాబితా చివర్లో ఉండిపోయాయి, మరియు `reconcile()` ఇంకా యాదృచ్ఛిక వైఫల్యాలనే ఎదుర్కొంది.

Deep Dive 17 §12 lo సరిగ్గా ఇదే జరిగింది. **ఒక పరీక్ష విఫలమైతే, పరీక్ష కూడా అనుమానితురాలే.**

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	`debit` ఆపై `dispense`	మౌలిక క్రమం
§4 (విరుపు)	`Network` మూడు ముగింపులతో	DOWN మరియు TIMEOUT ఒకేలా కనిపిస్తాయి
§7 (విరుపు)	Idempotency key	₹3,000 కి ₹9,000 debit
§8	`reverse` కూడా idempotent	లేకపోతే అదే సమస్య అద్దంలో
§10 (విరుపు)	`Journal` + `reconcile`	సరిచేయలేని విషయం మాయమైంది
§11	TIMEOUT → `REVERSAL_PENDING`	అనిశ్చితిలో సురక్షితమైన వైపు
§12	Network ముందు తనిఖీలు	తర్వాతి దశ వెనక్కి తీసుకోలేనిది
§13	రెండు-system అచలం	ఒక్క సమీకరణం చాలదు

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Card, PIN, నోట్లు – వేగంగా. Network అవతలి కథే అసలు interview.

6 నిమి	**Clarify** – నిల్వ ఎక్కడ? timeout అయితే? reversal ఉందా?
6 నిమి	PIN · నోట్లు · పరిమితులు – **వేగంగా**
13 నిమి – Idempotency	retry ఎందుకు ప్రమాదకరం · key
13 నిమి – Journal	compensation · **reconciliation**
7 నిమి	అచలం · రెండు ATMs · forecasting

ఏమి తప్పక చెప్పాలి

1. **"Timeout ≠ వైఫల్యం"** (§4) – "a timeout means I don't know whether it happened." **ఇదే ఈ interview యొక్క కేంద్రం**, మరియు దీన్ని మొదటి ఐదు నిమిషాల్లో చెప్పండి.
2. **Retry ఒక్కటే double-debit కి కారణం** (§7) – ₹3,000 కి ₹9,000 ఉదాహరణ చెప్పండి.
3. **Idempotency key ఒక ఒప్పందం** (§8) – key **పంపేవాడు** పుట్టిస్తాడు, **అందుకునేవాడు** గుర్తుపెట్టుకుంటాడు.
4. **Reversal ఒక compensation, rollback కాదు** (§9) – ledger lo రెండు entries ఉంటాయి.
5. **Journal + reconcile** (§11) – "if I can't fix it now, it has to survive until I can." మరియు **TIMEOUT కూడా reversal కి వెళుతుంది**.

ఏమి వదిలేయాలి

- `CashBox.plan` **రాయొద్దు** – "bounded coin change, same DP as making change" అని చెప్పండి.
- **PIN, card blocking** – రెండు వాక్యాలు.
- **Bank యొక్క అంతర్గత భాగం** – అది ఒక interface.
- **నిజమైన EMV/ISO-8583 వివరాలు** – అడిగితేనే (§16).

15. నోటితో చెప్పాల్సిన English script

"First question, and everything follows from it: does the ATM hold the balance, or is the account on a bank server across a network? If it's on a server, then this isn't really a vending machine problem – it's a distributed transaction problem, and I'd design around that.

The obvious order is right: debit first, then dispense. If you dispense first and the debit fails, the bank has lost money. But now the interesting part. When the ATM calls the bank, there are three outcomes, not two. Success. Network down, meaning the request never arrived and nothing happened. And timeout – which means the request may well have arrived, the debit may well have happened, and only the response was lost. The ATM sees the same failure for the last two, but in one case the balance is untouched and in the other three thousand rupees are gone. A timeout isn't a failure; it's an absence of information.

That's what makes retries dangerous. Retrying a timeout feels obviously correct – it's what every HTTP client does. But if the debit did land, retrying debits again. I ran it: a customer asking for three thousand rupees, two timeouts then a success, got three thousand in cash and nine thousand taken from their account, and the ATM displayed 'transaction successful'. Retry assumes replaying an operation gives the same result. That's true for reads and completely false for moving money.

So every withdrawal gets an idempotency key, generated by the ATM once, outside the retry loop, and sent with every attempt. The bank records the result against that key and returns the same result for a repeat instead of debiting again. The important part is that it's a contract between two sides – the caller has to keep the key stable and the callee has to remember it. If either half is missing it doesn't work.

Next: the debit succeeded and then the notes jam. I can't roll back, because the balance lives in another system. What I can do is send a compensating reversal – and that has to be idempotent too, or retrying the reversal over-credits the customer. It's not a rollback; the original debit stays in the ledger and a reversing entry sits next to it.

And then the case people miss. The reversal can fail too. At that point the customer is out three thousand rupees, the ATM knows it, and the moment that function returns, that knowledge is gone. So the ATM writes a journal entry before it starts and marks each phase as it goes, and anything left in reversal-pending gets retried by a reconcile pass when the network comes back. I'd also route timeouts into reversal-pending rather than guessing – if the debit happened, the reversal fixes it; if it didn't, the bank says there's nothing to reverse, which is harmless. When you're uncertain, err toward the customer.

One small thing that matters a lot: everything I can check without the network – amount validity, daily limit, whether the note denominations can even make this amount – I check before writing anything or calling anyone. The next step is the one I can't take back.

For testing I'd use an invariant rather than cases: money out of the account equals cash

dispensed plus unresolved liabilities. I ran two thousand ATMs with a twenty-five percent network failure rate and twenty percent jams – about two crore rupees dispensed – and after reconciliation there were zero unresolved transactions and the ledger, the balance, the cash box and the journal all agreed."

16. Follow-ups – రెండు ATMs, cash forecasting, EMV

Follow-up	జవాబు	మారే classes
"ATM పునఃప్రారంభమైతే?"	Journal **disk మీద** ఉండాలి; startup lo `reconcile()`	Storage
"రెండు ATMs, ఒకే ఖాతా, ఒకేసారి"	Bank వైపు atomic debit – Deep Dive 04 §7	Bank
"Deposit కూడా"	అదే నమూనా తిరగబడి – నోట్లు ముందు, credit తర్వాత	**+1 method**
"బదిలీ (transfer)"	రెండు debits/credits – ఒక పూర్తి saga	Bank
"ఏ ATM lo ఎంత నగదు ఉందో"	Deep Dive 19 యొక్క counters + `cash.total`	**0 concepts**
"ఎప్పుడు నగదు నింపాలి"	కింద చూడండి	Forecasting layer
"నిజమైన ATM protocol"	కింద చూడండి	–

నగదు అంచనా

> *"ఒక ATM ఖాళీ అయితే అది **కనిపించని వైఫల్యం** – Deep Dive 20 §16 lo చెప్పిన అదే ఆకారం.*
>
> *కానీ ఇక్కడ ఒక అదనపు మలుపు ఉంది: **ఏ నోట్లు అయిపోతాయో ముఖ్యం**. ₹2000 నోట్లు ఉండి ₹100 నోట్లు అయిపోతే, ATM lo ₹50,000 ఉన్నా అది **₹1,300 ఇవ్వలేదు**.*
>
> *కాబట్టి అంచనా వేయాల్సినది మొత్తం కాదు – **ఒక్కో denomination**. మరియు అది అడిగే మొత్తాల నమూనా మీద ఆధారపడుతుంది: ₹500 గుణిజాలు ఎక్కువ అడిగితే ₹500 నోట్లు వేగంగా అయిపోతాయి.*
>
> ***నా `plan()` ఇప్పటికే ఒక telemetry సంకేతం ఇస్తోంది** – అది `null` తిరిగి ఇచ్చిన ప్రతిసారీ, ఆ మొత్తం మరియు ఆ క్షణపు నోట్ల మిశ్రమం నమోదు చేస్తే, ఏ denomination కొరత ఖర్చు పెడుతోందో తెలుస్తుంది."*

నిజమైన protocol

> *"నిజమైన ATMs **ISO 8583** అనే సందేశ ప్రమాణం వాడతాయి, మరియు అందులో ఈ doc lo చూసినవన్నీ **పేర్లతో సహా** ఉన్నాయి:*
>
> - ***Retrieval Reference Number (RRN)*** *– నా* `key`*. ఒక్కో లావాదేవీకి ఒక్కటి, retries అన్నిటికీ అదే.*
> - ***Reversal messages (0400/0420)*** *– నా* `reverse`*. మరియు ప్రమాణం స్పష్టంగా చెప్తుంది: reversal **జవాబు వచ్చేదాకా పంపుతూనే ఉండాలి**.*
> - ***Stand-in processing*** *– bank చేరకపోతే ATM ఒక పరిమితి వరకు తనే నిర్ణయించడం, తర్వాత settle చేయడం.*
>
> ***ఇది ఒక ఆహ్లాదకరమైన నిర్ధారణ:*** *ఈ మూడు విరుపులూ ఊహించినవి కావు – 1980ల నుంచి బ్యాంకింగ్ ప్రమాణాలు వాటినే పరిష్కరిస్తూ వచ్చాయి. Interview lo ఆ పేర్లు చెప్పడం మీ జవాబుకి బరువు ఇస్తుంది."*

---

17. ఏమి నేర్చుకున్నాం

ఆలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
Timeout ≠ వైఫల్యం	DOWN vs TIMEOUT (§4)	ప్రతి network call, ప్రతి API
Retry safe కాదు	₹9,000 debit (§7)	Payments, emails, webhooks
Idempotency ఒక ఒప్పందం	Key పంపేవాడు పుట్టిస్తాడు (§8)	Stripe, PSPs, message queues
Compensation ≠ rollback	`reverse` (§9)	Sagas, microservices
సరిచేయలేకపోతే గుర్తుపెట్టుకో	Journal (§11)	WAL, outbox pattern, dead-letter queues
అనిశ్చితిలో సురక్షిత వైపు	TIMEOUT → reversal (§11)	Deep Dive 04 §9 · HELD
వెనక్కి తీసుకోలేని దశకి ముందు తనిఖీ	§12	Deep Dive 20 §12 · ప్రతి commit

ఒక చివరి ఆలోచన – ఈ PROBLEM ఎందుకు ప్రత్యేకమైనది

ఈ series lo **ఇరవై problems** lo ఇది మొదటిసారి – ఇక్కడ **సరైన పరిష్కారం అనేది లేదు**.

మిగతా అన్నిచోట్లా ఒక విరుపుని పూర్తిగా మూసేయగలిగాం: piece table అక్షరాలని కదపదు, trie prefix ని నడుస్తుంది, escrow నాణేలని గుర్తుపెడుతుంది. **సమస్య పోయింది.**

ఇక్కడ అది సాధ్యం కాదు. ATM ఎప్పటికీ ఖచ్చితంగా తెలుసుకోలేదు debit జరిగిందో లేదో. నేను చేసినదంతా – **ఆ అజ్ఞానాన్ని సురక్షితంగా మార్చడం**: మళ్ళీ అడిగినా హాని లేకుండా (idempotency), తప్పుని సరిచేయగలిగేలా (compensation), మరియు ఇప్పుడు కుదరకపోతే తర్వాత కుదిరేలా (reconciliation).

అదే distributed systems యొక్క నిజమైన స్వభావం: అనిశ్చితిని తొలగించడం కాదు – **దానితో కలిసి జీవించడం**.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot** · **02 Cache** · **03 Rate Limiter** · **04 BookMyShow** · **05 Splitwise** · **06 Elevator** · **07 Pub-Sub** · **08 HashMap** · **09 Chess** · **10 Meeting Scheduler** · **11 Food Delivery** · **12 File System** · **13 Leaderboard** · **14 Text Editor** · **15 Library** · **16 Tic-Tac-Toe** · **17 Autocomplete** · **18 Snake & Ladder** · **19 Hit Counter** · **20 Vending Machine** · **21 ATM**.

Deep Dive 20 (Vending Machine) ఈ doc కి తప్పనిసరి ముందుమాట – అక్కడ ఒకే పెట్టెలో డబ్బు, ఇక్కడ network అవతల.

ATM – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output, ₹1.96 కోట్ల అనుకరణతో సహా నిజంగా `node` lo run చేసి తీసినదే ✅